

UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN



CURSO:

Algoritmos y Estructura de Datos

CICLO:

IV

DOCENTE:

Fernández Bejarano Raul Enrique

ALUMNO:

Cerrón Villar Maricielo Sarai

HUANCAYO – PERÚ

2026

ESTRUCTURAS FUNDAMENTALES

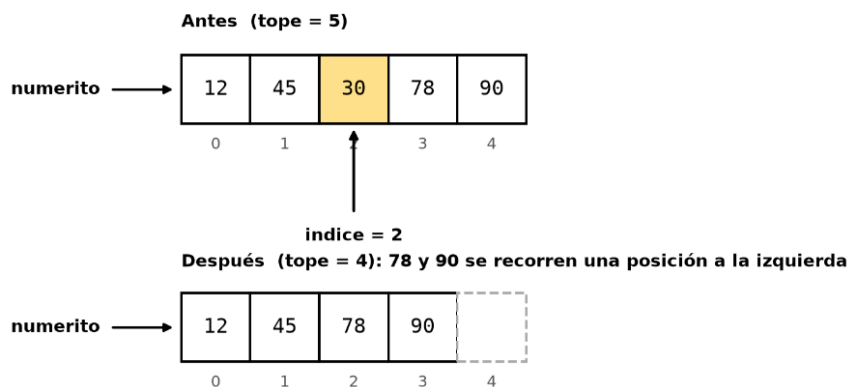
ACTIVIDAD 1:

1: Elaborar un método para eliminar un elemento “x”, del arreglo creado anteriormente.

Este método busca la primera ocurrencia de un valor y lo retira del arreglo sin dejar huecos vacíos.

¿Cómo se aplica?

1. Se recorre el arreglo desde el inicio $i = 0$ hasta $\text{tope} - 1$. Si se encuentra que $\text{numerito}[i] == x$, se guarda esa posición en una variable pos y se interrumpe la búsqueda con `break`.
2. Se verifica si el elemento fue encontrado ($\text{pos} \neq -1$). Si no existe, se muestra un mensaje de alerta.
3. Si se encontró, se inicia un bucle desde pos hasta $\text{tope} - 2$. Cada casilla toma el valor de la siguiente: $\text{numerito}[i] = \text{numerito}[i + 1]$.
4. Se resta 1 a la variable tope ($\text{tope}--$), reduciendo el tamaño lógico del arreglo.



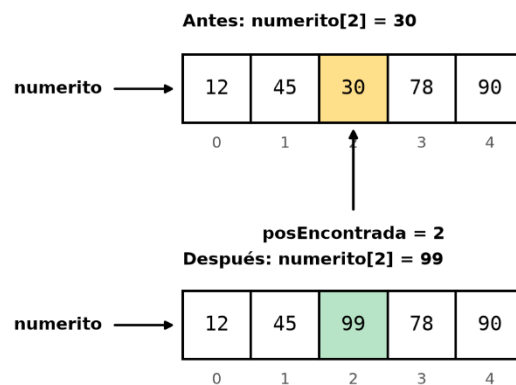
```
public void eliminarElemento() {  
    if(tope == 0) { System.out.println("El arreglo esta vacio."); return; }  
    System.out.print("Ingrese el elemento a eliminar: ");  
    int x = teclado.nextInt();  
    int pos = -1;  
  
    for (int i = 0; i < tope; i++) {  
        if (numerito[i] == x) {  
            pos = i;  
            break;  
        }  
    }  
  
    if (pos != -1) {  
        for (int i = pos; i < tope - 1; i++) {  
            numerito[i] = numerito[i + 1];  
        }  
        tope--;  
        System.out.println("Elemento " + x + " eliminado.");  
    } else {  
        System.out.println("No existe el elemento en el arreglo.");  
    }  
}
```

2: Elaborar un método para modificar un elemento "x".

Este método busca reemplazar el valor de un elemento existente por uno nuevo.

¿Cómo se aplica?

1. Se inicia un bucle for para examinar el arreglo elemento por elemento desde $i = 0$ hasta $\text{tope} - 1$.
2. En cada iteración se evalúa si `numerito[i]` es igual a `x`.
3. Si coincide, se sobrescribe la casilla asignándole el nuevo valor: `numerito[i] = nuevoValor`.
4. Se marca una bandera de control (`encontrado = true`), se rompe el ciclo con `break` y se imprime un mensaje de aviso.



Código:

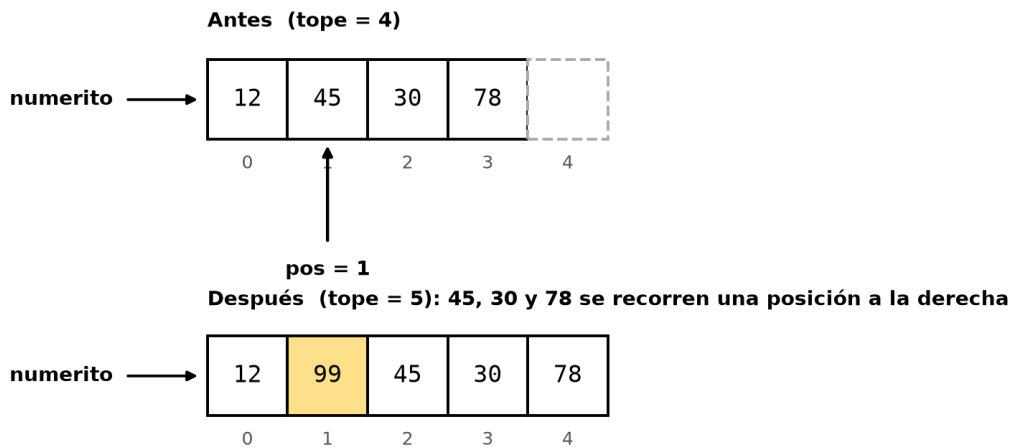
```
public void modificarElemento() {  
    if(tope == 0) { System.out.println("El arreglo esta vacio."); return; }  
    System.out.print("Ingrese el elemento a modificar: ");  
    int x = teclado.nextInt();  
    boolean encontrado = false;  
  
    for (int i = 0; i < tope; i++) {  
        if (numerito[i] == x) {  
            System.out.print("Ingrese el nuevo valor: ");  
            numerito[i] = teclado.nextInt();  
            encontrado = true;  
            System.out.println("Elemento modificado.");  
            break;  
        }  
    }  
    if (!encontrado) {  
        System.out.println("No existe el elemento.");  
    }  
}
```

3: Elaborar un método para insertar un elemento "x" en una posición "y".

Este método busca colocar un nuevo dato en un índice específico desplazando los demás elementos para hacer espacio.

¿Cómo se aplica?

1. Se verifica que la posición y sea válida ($y \geq 0$ y $y \leq \text{tope}$) y que el arreglo no esté lleno ($\text{tope} < \text{numerito.length}$).
2. Se recorre el arreglo hacia atrás, desde $i = \text{tope}$ bajando hasta $y + 1$. Cada casilla recibe el valor de la izquierda: $\text{numerito}[i] = \text{numerito}[i - 1]$.
3. Al liberar la casilla y, se le asigna el nuevo valor: $\text{numerito}[y] = x$.
4. Se incrementa en 1 la variable tope ($\text{tope}++$).



Código:

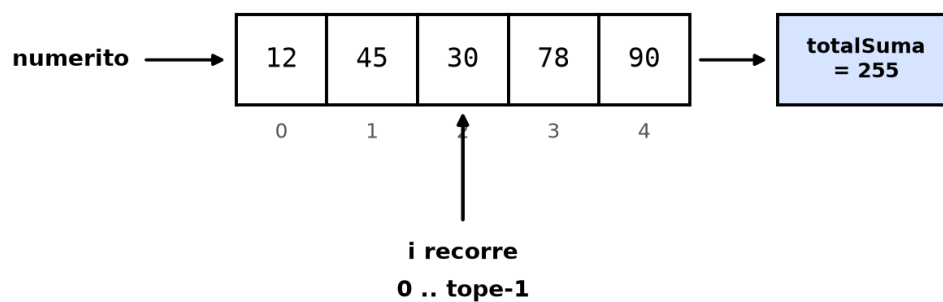
```
public void insertarElemento() {  
    if (tope < numerito.length) {  
        System.out.print("Ingrese el nuevo elemento a insertar: ");  
        int x = teclado.nextInt();  
        System.out.print("Ingrese la posicion (0 a " + tope + "): ");  
        int y = teclado.nextInt();  
  
        if (y >= 0 && y <= tope) {  
            for (int i = tope; i > y; i--) {  
                numerito[i] = numerito[i - 1];  
            }  
            numerito[y] = x;  
            tope++;  
            System.out.println("Elemento insertado.");  
        } else {  
            System.out.println("Posicion no valida.");  
        }  
    } else {  
        System.out.println("El arreglo esta lleno.");  
    }  
}
```

4: Elaborar un método para sumar los elementos del arreglo.

Este método busca acumular el total de todos los datos almacenados.

¿Cómo se aplica?

1. Se crea una variable acumuladora suma asignándole 0.
2. Se itera mediante un bucle for desde el índice 0 hasta tope - 1.
3. En cada paso se añade el valor de la casilla actual a la variable: $\text{suma} = \text{suma} + \text{numerito}[i]$.
4. Al terminar el ciclo, se imprime el valor final retenido en suma.



Código:

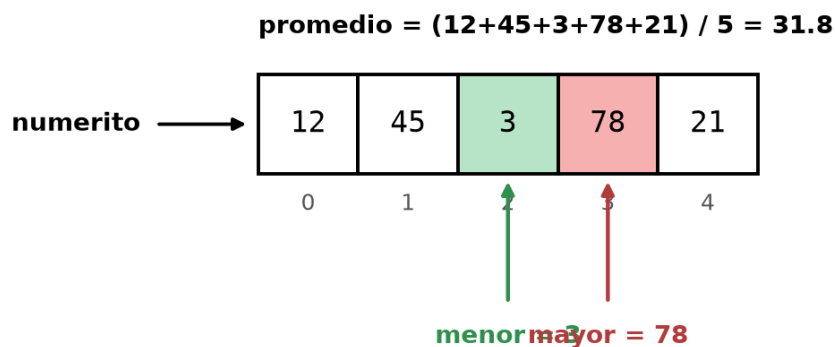
```
public void sumarElementos() {  
    int suma = 0;  
    for (int i = 0; i < tope; i++) {  
        suma = suma + numerito[i];  
    }  
    System.out.println("La suma de todos los elementos es: " + suma);  
}
```

5: Elaborar un método para hallar el número menor, el número mayor y el promedio de los elementos del arreglo.

Este método busca generar un reporte de métricas clave sobre el conjunto de datos.

¿Cómo se aplica?

1. Si `tope == 0`, se notifica que el arreglo está vacío y finaliza el método.
2. Se asume temporalmente que el primer elemento `numerito[0]` es tanto el menor como el mayor. La variable `suma` se inicia en 0.
3. Se recorre el arreglo del índice 0 a `tope - 1`:
 - Si `numerito[i] < menor`, se actualiza `menor = numerito[i]`.
 - Si `numerito[i] > mayor`, se actualiza `mayor = numerito[i]`.
 - Se suma el elemento a la variable acumuladora.
4. Se calcula dividiendo la suma total entre el número de elementos: `promedio = suma / tope`.



Código:

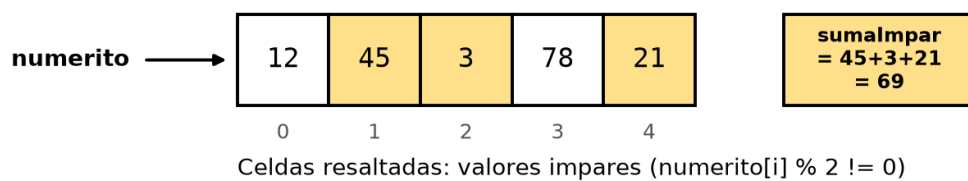
```
public void hallarMenorMayorPromedio() {  
    if (tope > 0) {  
        int menor = numerito[0];  
        int mayor = numerito[0];  
        int suma = 0;  
  
        for (int i = 0; i < tope; i++) {  
            if (numerito[i] < menor) menor = numerito[i];  
            if (numerito[i] > mayor) mayor = numerito[i];  
            suma = suma + numerito[i];  
        }  
  
        double promedio = (double) suma / tope;  
        System.out.println("Numero menor: " + menor);  
        System.out.println("Numero mayor: " + mayor);  
        System.out.println("Promedio: " + promedio);  
    } else {  
        System.out.println("No existen elementos.");  
    }  
}
```

6: Elaborar un método para suma de impares los elementos del arreglo.

Este método busca filtrar los números impares y calcular su suma total.

¿Cómo se aplica?

1. Se declara la variable suma = 0.
2. Se recorren las casillas ocupadas del arreglo. En cada iteración se evalúa mediante la operación módulo % si el número es impar ($\text{numerito}[i] \% 2 \neq 0$).
3. Solo si la condición de imparidad es verdadera, el número se suma a la variable suma.
4. Se muestra la suma total obtenida al finalizar el bucle.



Código:

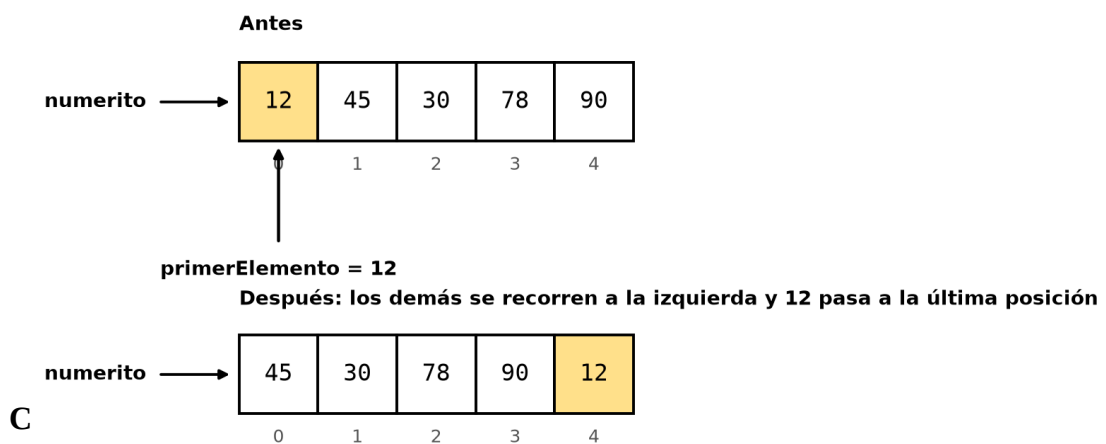
```
public void sumaImpares() {  
    int suma = 0;  
    for (int i = 0; i < tope; i++) {  
        if (numerito[i] % 2 != 0) {  
            suma = suma + numerito[i];  
        }  
    }  
    System.out.println("La suma de los elementos impares es: " + suma);  
}
```

7: Elaborar un método para mover el primer elemento a la última posición del arreglo.

Este método busca rotar el arreglo enviando el primer dato al final y corriendo el resto hacia la izquierda.

¿Cómo se aplica?

1. Si hay más de un elemento ($\text{tope} > 1$), se guarda el valor del primer casillero en una variable auxiliar: `int primero = numerito[0]`.
2. Se recorre el arreglo desde $i = 0$ hasta $\text{tope} - 2$, moviendo cada valor una casilla a la izquierda: `numerito[i] = numerito[i + 1]`.
3. Se coloca el valor de `primero` en la última casilla ocupada: `numerito[tope - 1] = primero`.



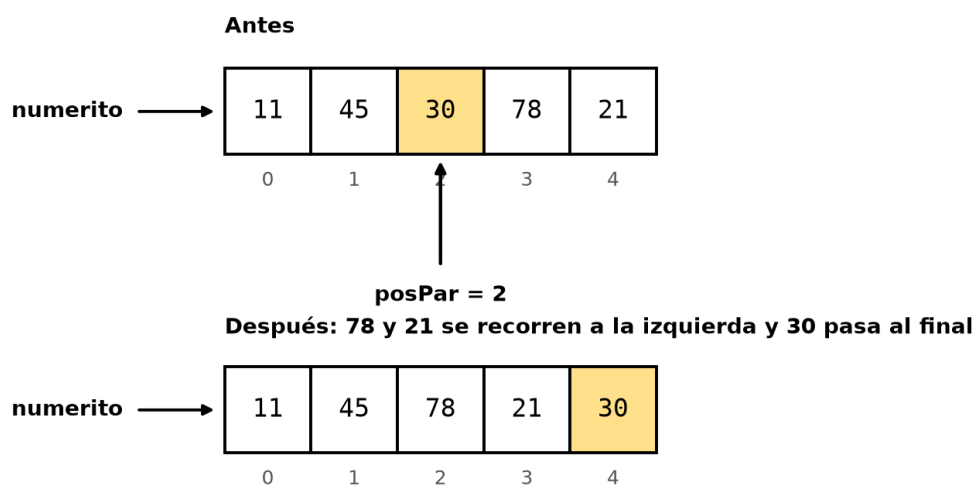
```
public void moverPrimerAlFinal() {
    if (tope > 1) {
        int aux = numerito[0];
        for (int i = 0; i < tope - 1; i++) {
            numerito[i] = numerito[i + 1];
        }
        numerito[tope - 1] = aux;
        System.out.println("Primer elemento movido al final.");
    } else {
        System.out.println("Se necesitan al menos 2 elementos.");
    }
}
```


8: Elaborar un método para mover el primer elemento par al final del arreglo.

Este método busca el primer número par dentro de la lista y trasladarlo a la última posición ocupada.

¿Cómo se aplica?

1. Se recorre el arreglo buscando el primer elemento que cumpla $\text{numerito}[i] \% 2 == 0$. Se guarda su posición en `posPar` y se detiene la búsqueda.
2. Si se encontró un par, se guarda su valor en `valorPar`. Luego, se recorren las casillas desde `posPar` hasta `tope - 2`, desplazando los elementos una posición hacia la izquierda para cubrir la casilla vacía.
3. Se asigna `valorPar` en la posición `numerito[tope - 1]`.



```
public void primerParAlFinal() {  
    if (tope > 0) {  
        boolean movido = false;  
        for (int i = 0; i < tope - 1; i++) {  
            if (numerito[i] % 2 == 0) {  
                int aux = numerito[i];  
                for (int j = i; j < tope - 1; j++) {  
                    numerito[j] = numerito[j+1];  
                }  
                numerito[tope - 1] = aux;  
                System.out.println("Primer par movido al final.");  
                movido = true;  
                break;  
            }  
        }  
        if (!movido) System.out.println("No se encontraron pares para mover (o ya esta al final).");  
    }  
}
```

ACTIVIDAD 2:

Desarrollar una solución que permita ingresar y almacenar las notas de todas las prácticas de un alumno, determinar el promedio de las prácticas sabiendo que se elimina la práctica con la nota más baja, mostrar el promedio redondeado, validar el ingreso de datos.

El propósito de este programa es calcular la nota final del alumno descartando su peor calificación y aplicando redondeo.

¿Cómo se aplica?

1. Mediante un ciclo do-while, se solicita la cantidad de prácticas y se exige que sea al menos 2
2. Se crea el arreglo notas de tamaño igual a la cantidad ingresada. Con un bucle for, se pide cada nota dentro de un ciclo do-while interno que valida que el valor se encuentre estrictamente entre 0 y 20
3. Se inicializa menor con la primera nota (notas[0]).
4. Se recorre el arreglo sumando cada nota a sumaTotal y actualizando menor si se encuentra una calificación más baja.
5. Se resta la peor nota a la suma general ($\text{sumaFinal} = \text{sumaTotal} - \text{menor}$) y se divide entre (cantidad - 1)
6. Se aplica la función `Math.round(promedio)` para obtener el entero representativo y se muestran los resultados en pantalla.

```

package PromedioPracticas;
import java.util.Scanner;

public class PromedioPracticas {

    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.println("\tSISTEMA DE PROMEDIO DE NOTAS");
        System.out.println("\t=====");

        int numPracticas;

        do {
            System.out.print("Ingrese la cantidad total de practicas (minimo 2): ");
            numPracticas = teclado.nextInt();
            if(numPracticas < 2) {
                System.out.println("Error: Debe ingresar al menos 2 practicas para poder eliminar una.\n");
            }
        } while (numPracticas < 2);

        double[] notas = new double[numPracticas];

        System.out.println("\n\tINGRESO DE NOTAS");
        System.out.println("\t=====");

        for (int i = 0; i < notas.length; i++) {
            do {
                System.out.print("Ingrese la nota de la practica " + (i + 1) + " (0 - 20): ");
                notas[i] = teclado.nextDouble();

                if (notas[i] < 0 || notas[i] > 20) {
                    System.out.println(">> Nota no valida. Por favor, ingrese un valor entre 0 y 20.");
                }
            } while (notas[i] < 0 || notas[i] > 20);
        }

        double notaMenor = notas[0];
        double suma = 0;

        for (int i = 0; i < notas.length; i++) {
            if (notas[i] < notaMenor) {
                notaMenor = notas[i];
            }
            suma = suma + notas[i];
        }

        double sumaFinal = suma - notaMenor;
        double promedio = sumaFinal / (numPracticas - 1);

        System.out.println("\n\tRESULTADOS FINALES");
        System.out.println("\t=====");

        System.out.print("Notas ingresadas: ");
        for (int i = 0; i < notas.length; i++) {
            System.out.print "[" + notas[i] + " ] ";
        }
        System.out.println("\n");

        System.out.println("Nota mas baja (eliminada) : " + notaMenor);
        System.out.println("Suma de practicas validas : " + sumaFinal);
        System.out.println("Promedio exacto : " + promedio);
        System.out.println("Promedio redondeado : " + Math.round(promedio));
    }
}

```

RESULTADO:

```
run:
    SISTEMA DE PROMEDIO DE NOTAS
    =====
Ingrese la cantidad total de practicas (minimo 2): 3

    INGRESO DE NOTAS
    =====
Ingrese la nota de la practica 1 (0 - 20): 15
Ingrese la nota de la practica 2 (0 - 20): 18
Ingrese la nota de la practica 3 (0 - 20): 20

    RESULTADOS FINALES
    =====
Notas ingresadas: [15.0] [18.0] [20.0]

Nota mas baja (eliminada) : 15.0
Suma de practicas validas : 38.0
Promedio exacto           : 19.0
Promedio redondeado       : 19
BUILD SUCCESSFUL (total time: 25 seconds)
```